

NLP Course

Search documentation

Ctrl+K

EN

2,357

0. SETUP

1. TRANSFORMER MODELS

2. USING TRANSFORMERS

Introduction

Behind the pipeline

Models

Tokenizers

Handling multiple sequences

Putting it all together

Basic usage completed!

End-of-chapter quiz

3. FINE-TUNING A PRETRAINED MODEL

4. SHARING MODELS AND TOKENIZERS

5. THE DATASETS LIBRARY

6. THE TOKENIZERS LIBRARY

7. MAIN NLP TASKS

8. HOW TO ASK FOR HELP

9. BUILDING AND SHARING DEMOS

COURSE EVENTS

Join the Hugging Face community

and get access to the augmented documentation experience

Collaborate on models, datasets and Spaces

Faster examples with accelerated inference

Switch between documentation themes

Sign Up

to get started

Pytorch

TensorFlow

Ask a question

Open in Colab

Open In Studio Lab

Models

Instantiate a Transformers model (PyTorch)

In this section we'll take a closer look at creating and using a model. We'll use the `AutoModel` class, which is handy when you want to instantiate any model from a checkpoint.

The `AutoModel` class and all of its relatives are actually simple wrappers over the wide variety of models available in the library. It's a clever wrapper as it can automatically guess the appropriate model architecture for your checkpoint, and then instantiates a model with this architecture.

However, if you know the type of model you want to use, you can use the class that defines its architecture directly. Let's take a look at how this works with a BERT model.

Creating a Transformer

The first thing we'll need to do to initialize a BERT model is load a configuration object:

```
from transformers import BertConfig, BertModel

# Building the config
config = BertConfig()

# Building the model from the config
model = BertModel(config)
```

The configuration contains many attributes that are used to build the model:

```
print(config)

BertConfig {
  [...]
  "hidden_size": 768,
  "intermediate_size": 3072,
  "max_position_embeddings": 512,
  "num_attention_heads": 12,
  "num_hidden_layers": 12,
  [...]
}
```

While you haven't seen what all of these attributes do yet, you should recognize some of them: the `hidden_size` attribute defines the size of the `hidden_states` vector, and `num_hidden_layers` defines the number of layers the Transformer model has.

Different loading methods

Creating a model from the default configuration initializes it with random values:

```
from transformers import BertConfig, BertModel

config = BertConfig()
model = BertModel(config)

# Model is randomly initialized!
```

The model can be used in this state, but it will output gibberish; it needs to be trained first. We could train the model from scratch on the task at hand, but as you saw in [Chapter 1](#), this would require a long time and a lot of data, and it would have a non-negligible environmental impact. To avoid unnecessary and duplicated effort, it's imperative to be able to share and reuse models that have already been trained.

Loading a Transformer model that is already trained is simple — we can do this using the `from_pretrained()` method:

```
from transformers import BertModel

model = BertModel.from_pretrained("bert-base-cased")
```

As you saw earlier, we could replace `BertModel` with the equivalent `AutoModel` class. We'll do this from now on as this produces checkpoint-agnostic code; if your code works for one checkpoint, it should work seamlessly with another. This applies even if the architecture is different, as long as the checkpoint was trained for a similar task (for example, a sentiment analysis task).

In the code sample above we didn't use `BertConfig`, and instead loaded a pretrained model via the `bert-base-cased` identifier. This is a model checkpoint that was trained by the authors of BERT themselves; you can find more details about it in its [model card](#).

This model is now initialized with all the weights of the checkpoint. It can be used directly for inference on the tasks it was trained on, and it can also be fine-tuned on a new task. By training with pretrained weights rather than from scratch, we can quickly achieve good results.

The weights have been downloaded and cached (so future calls to the `from_pretrained()` method won't re-download them) in the cache folder, which defaults to `~/.cache/huggingface/transformers`. You can customize your cache folder by setting the `HF_HOME` environment variable.

The identifier used to load the model can be the identifier of any model on the Model Hub, as long as it is compatible with the BERT architecture. The entire list of available BERT checkpoints can be found [here](#).

Saving methods

Saving a model is as easy as loading one — we use the `save_pretrained()` method, which is analogous to the `from_pretrained()` method:

```
model.save_pretrained("directory_on_my_computer")
```

This saves two files to your disk:

```
ls directory_on_my_computer

config.json  pytorch_model.bin
```

If you take a look at the `config.json` file, you'll recognize the attributes necessary to build the model architecture. This file also contains some metadata, such as where the checkpoint originated and what  Transformers version you were using when you last saved the checkpoint.

The `pytorch_model.bin` file is known as the *state dictionary*; it contains all your model's weights. The two files go hand in hand; the configuration is necessary to know your model's architecture, while the model weights are your model's parameters.

Using a Transformer model for inference

Now that you know how to load and save a model, let's try using it to make some predictions. Transformer models can only process numbers — numbers that the tokenizer generates. But before we discuss tokenizers, let's explore what inputs the model accepts.

Tokenizers can take care of casting the inputs to the appropriate framework's tensors, but to help you understand what's going on, we'll take a quick look at what must be done before sending the inputs to the model.

Let's say we have a couple of sequences:

```
sequences = ["Hello!", "Cool.", "Nice!"]
```

The tokenizer converts these to vocabulary indices which are typically called *input IDs*. Each sequence is now a list of numbers! The resulting output is:

```
encoded_sequences = [
  [101, 7592, 999, 102],
  [101, 4658, 1012, 102],
  [101, 3835, 999, 102],
]
```

This is a list of encoded sequences: a list of lists. Tensors only accept rectangular shapes (think matrices). This "array" is already of rectangular shape, so converting it to a tensor is easy:

```
import torch

model_inputs = torch.tensor(encoded_sequences)
```

Using the tensors as inputs to the model

Making use of the tensors with the model is extremely simple — we just call the model with the inputs:

```
output = model(model_inputs)
```

While the model accepts a lot of different arguments, only the input IDs are necessary. We'll explain what the other arguments do and when they are required later, but first we need to take a closer look at the tokenizers that build the inputs that a Transformer model can understand.

Models

Creating a Transformer

Different loading methods

Saving methods

Using a Transformer model for inference

Using the tensors as inputs to the model

